

UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE INGENIERÍA — DEPARTAMENTO DE INGENIERÍA DE SISTEMAS E INDUSTRIAL
ASIGNATURA: LENGUAJES DE PROGRAMACIÓN (2026-2)

REPORTE TÉCNICO FORMAL — TAREA 17: ARITMÉTICA DE PUNTO FLOTANTE IEEE 754 (BINARY64) PARA EL COMPUTADOR ENIGMA-64

Emulador de FPU en Ensamblador Puro, Mesa Canónica de Vectores, Estimación de Raíz Cuadrada por
Newton-Raphson, Constante de Brun y Batería Integral de Pruebas

EQUIPO DE DESARROLLO (AUTORES):

Tomás Felipe Garzón Gómez — Integrante 1 (FPU Núcleo & FADD/FSUB)

Juan Sebastián Umaña Camacho — Integrante 2 (FPU Multiplicación FMUL de 128 bits)

Michael Stiven Betancourt Gelves — Integrante 3 (FPU División FDIV & Comparador FCMP)

Deiby Santiago Barragán Gaitán — Integrante 4 (FPU Conversiones INT/FLOAT & Mesa de Vectores)

Maicol Sebastián Olarte Ramírez — Integrante 5 (GUI & Panel Interactivo FPU en Tkinter)

Juan Luis Vergara Novoa — Integrante 6 (Algoritmo Raíz Cuadrada Newton-Raphson & Oráculo)

Alejandro Argüello Muñoz — Integrante 7 (Constante de Brun & Batería Completa de Pruebas)

DOCENTE TITULAR:

Jorge Eduardo Ortiz Triviño (Profesor Asociado DISI)

Bogotá D.C., Colombia — Octubre de 2026

1. MARCO TEÓRICO

El computador Enigma-64, concebido como una arquitectura Von Neumann de 64 bits con memoria físicamente paginada y palabra natural de 8 bytes, incorpora en la Tarea 17 una **Unidad de Punto Flotante (FPU)** completa implementada a nivel de software mediante lenguaje ensamblador puro. El diseño se ciñe rigurosamente a la norma internacional **IEEE 754-2008** para el formato de doble precisión binaria (**Binary64**).

1.1. Representación de Números en Punto Flotante IEEE 754 Doble Precisión (Binary64)

En el formato Binary64, una palabra de 64 bits se descompone en tres campos canónicos contiguos:

- **Signo (s, bit 63):** 1 bit donde 0 representa valores positivos y 1 denota valores negativos.
- **Exponente sesgado (e, bits 62..52):** 11 bits con un sesgo (*bias*) fijo de **1023**, abarcando exponentes no sesgados en el rango $[-1022, +1023]$.
- **Mantisa fraccionaria (f, bits 51..0):** 52 bits explícitos. Para números normales, existe un bit implícito unitario (1.f), totalizando 53 bits de precisión efectiva (~ 15.9 dígitos decimales).

Clase de Número	Exponente (e)	Fracción (f)	Valor Matemático Representado	Ejemplo Hexadecimal (64b)
Cero (+0 / -0)	0 (todos 0)	0 (todos 0)	$(-1)^s \times 0.0$	0x0000000000000000
Subnormal	0 (todos 0)	$f \neq 0$	$(-1)^s \times 2^{-1} \times (0.f)$	0x0000000000000001
Normal	$1 \leq e \leq 2046$	Cualquiera	$(-1)^s \times 2^{e-1023} \times (1.f)$	0x3FF0000000000000 (+1.0)
Infinito ($+\infty$ / $-\infty$)	2047 (todos 1)	0 (todos 0)	$(-1)^s \times \infty$	0x7FF0000000000000 ($+\infty$)
NaN (Not a Number)	2047 (todos 1)	$f \neq 0$	Indeterminado (qNaN / sNaN)	0x7FF8000000000000 (qNaN)

Tabla 1: Especificación de campos, sesgos y casos especiales en IEEE 754 Binary64.

1.2. Algoritmos de Aritmética Flotante Básica en Ensamblador Puro

Dado que la ALU de Enigma-64 únicamente ejecuta instrucciones de números enteros de 64 bits, las operaciones flotantes fueron sintetizadas mediante algoritmos canónicos de descomposición y recomposición:

1. **Suma y Resta (FADD / FSUB):** Desempaquetado de operandos, igualación de exponentes mediante corrimiento a la derecha de la mantisa de menor magnitud, suma/resta con signo de las mantisas alineadas, normalización por corrimiento a la izquierda/derecha y redondeo formal hacia el par más cercano (*roundTiesToEven*).
2. **Multipliación (FMUL):** Signo resultante por XOR ($S = S_A \oplus S_B$). Suma de exponentes con corrección de sesgo ($E_R = E_A + E_B - 1023$). Multipliación completa de mantisas de 53×53 bits (producto de hasta 106 bits) descompuesta en cuatro multiplicaciones de 32 bits en la subrutina FPU_MUL128, con extracción rigurosa de bits Guard, Round y Sticky.
3. **División (FDIV):** Signo $S = S_A \oplus S_B$, resta de exponentes ($E_R = E_A - E_B + 1023$). División larga entera binaria de mantisas de 64 bits con detección de residuo exacto para bit Sticky y detección formal de división por cero.
4. **Comparación (FCMP):** Distinción de NaNs (retorna código no ordenado 2), orden estricto de signos, comparación de magnitudes y reconocimiento de equivalencia $+0.0 == -0.0$ (retorna 0).
5. **Conversiones (FPU_INT_TO_FLOAT / FPU_FLOAT_TO_INT):** Detección de signo, búsqueda del MSB, empaquetado sesgado y truncamiento.

1.3. Método Numérico de Newton-Raphson para Raíz Cuadrada (Ricardo Peña, Pág. 26)

En la página 26 del libro *De Euclides a JAVA: historia de los algoritmos y de los lenguajes de programación* (Ricardo Peña, 2006), se documenta el método iterativo clásico para la extracción de la raíz cuadrada de un número real $A > 0$ resolviendo la ecuación no lineal $f(x) = x^2 - A = 0$. Aplicando la derivada $f'(x) = 2x$:

$$x_{k+1} = \frac{1}{2} \cdot (x_k + A / x_k)$$

El método goza de convergencia cuadrática (duplica la cantidad de cifras exactas en cada ciclo). Para garantizar convergencia en 4–5 iteraciones sobre punto flotante de 64 bits, se calcula analíticamente la semilla inicial a partir del exponente IEEE 754: $e_0 = \lfloor (E_A - 1023) / 2 \rfloor$, ensamblando $x_0 = (e_0 + 1023) \ll 52$.

1.4. Teoría y Convergencia de la Constante de Brun (B■)

En 1919, el matemático noruego Viggo Brun demostró que la suma de los recíprocos de los números primos gemelos (pares de primos de la forma $(p, p+2)$) converge hacia una constante matemática finita denominada **Constante de Brun (B■)**:

$$B■ = \sum_{(p, p+2) \in \mathbb{N}} (1/p + 1/(p+2)) = (1/3 + 1/5) + (1/5 + 1/7) + (1/11 + 1/13) + \dots \approx 1.90216058\dots$$

A diferencia de la serie armónica clásica de los números primos que diverge, la serie de Brun converge de forma sumamente lenta. Desde la óptica de la ingeniería computacional, la estimación de $B■$ representa un banco de pruebas de esfuerzo extremo (*stress test*): requiere ejecutar miles de divisiones flotantes ($1/p$ y $1/(p+2)$), acumulaciones repetitivas con redondeo inexacto y cientos de llamadas anidadas a subrutinas evaluando la estabilidad absoluta del puntero de pila.

2. DESCRIPCIÓN Y JUSTIFICACIÓN DEL PROBLEMA

El computador Enigma-64 fue concebido originalmente con un repertorio ISA restringido exclusivamente a enteros. Para posibilitar la ejecución de aplicaciones científicas, algoritmos iterativos y la futura integración de un compilador de alto nivel, es imprescindible incorporar capacidades de punto flotante de doble precisión sin alterar el hardware físico base.

2.1. Desafíos Técnicos y Restricciones de Microarquitectura

- Ausencia de FPU por Hardware:** La CPU sólo dispone de suma, resta, multiplicación y división entera de 64 bits. Toda la manipulación de campos de bits, normalización y redondeo debe ser orquestada instrucción por instrucción en ensamblador.
- Preservación de Marcos de Pila en Llamadas Anidadas:** La evaluación de la Constante de Brun requiere que una función (FPU_BRUN) invoque consecutivamente a FPU_ES_PRIMO, FPU_INT_TO_FLOAT, FDIV y FADD. Dado que las convenciones de llamada establecen que los registros R1..R5 son volátiles (*caller-saved*), es imperativo implementar una gestión estricta del marco de pila (*Stack Frame*) mediante ENTER y LEAVE.
- Mesa Canónica de Vectores Desacoplada:** Para permitir que programas de usuario ejecuten servicios de la FPU sin conocer las direcciones de memoria internas de los autores, se requiere una tabla de salto fija (*Jump Table*) de 9 entradas canónicas.
- Fidelidad Absoluta al Estándar IEEE 754:** La emulación debe gestionar minuciosamente casos límite: división por cero (retornando infinito con signo XOR), ceros con signo (donde $+0.0 \neq -0.0$ pero $1/(+0) = +\infty$ y $1/(-0) = -\infty$), indeterminaciones ($0/0$ y ∞/∞ produciendo NaN canónico) y comparaciones no ordenadas con NaNs.

3. DISEÑO DE LA SOLUCIÓN Y ARQUITECTURA DEL SISTEMA

La solución se estructuró mediante un modelo modular multicapa donde cada integrante asumió un componente canónico, integrados finalmente a través de la Mesa de Vectores en el archivo consolidado programas/fpu_lib.s.

3.1. Estructura de la Mesa Canónica de Vectores (FPU_VECTORES)

Ubicada en el offset +0x00 de la biblioteca FPU cargada en memoria RAM, cada entrada consta exactamente de 5 bytes (1 byte de opcode JMP 0x15 + 4 bytes de dirección absoluta Big-Endian):

Vector	Offset	Instrucción Salto	Servicio Matemático IEEE 754	Integrante Responsable
VEC_FADD	+0x00	JMP FADD	Suma flotante IEEE 754 ($R5 = R1 + R2$)	Integrante 1
VEC_FSUB	+0x05	JMP FSUB	Resta flotante IEEE 754 ($R5 = R1 - R2$)	Integrante 1
VEC_FMUL	+0x0A	JMP FMUL	Multiplicación IEEE 754 ($R5 = R1 \times R2$)	Integrante 2
VEC_FDIV	+0x0F	JMP FDIV	División IEEE 754 ($R5 = R1 / R2$)	Integrante 3
VEC_FCMP	+0x14	JMP FCMP	Comparación de orden ($-1, 0, 1, 2$)	Integrante 3
VEC_INT_TO_FLOAT	+0x19	JMP FPU_INT_TO_FLOAT	Conversión Entero 64b $\rightarrow$ Float 64b	Integrante 4
VEC_FLOAT_TO_INT	+0x1E	JMP FPU_FLOAT_TO_INT	Conversión Float 64b $\rightarrow$ Entero truncado	Integrante 4
VEC_FSQRT	+0x23	JMP FSQRT	Raíz Cuadrada por Newton-Raphson	Integrante 6
VEC_FBRUN	+0x28	JMP FPU_BRUN	Estimación de Constante de Brun B■	Integrante 7

Tabla 2: Especificación formal de la Mesa de Vectores Canónicos (FPU_VECTORES).

3.2. Diseño Algorítmico de la Constante de Brun (FPU_BRUN)

La subrutina FPU_BRUN opera como un algoritmo numérico de orden superior que enlaza los módulos del equipo. El pseudocódigo formal de la subrutina implementada en ensamblador se detalla a continuación:

```
Subrutina FPU_BRUN(K_target):
  Crear marco de pila ENTER 128
  K_count = 0, p = 3, B2_acumulado = 0.0, const_1_0 = 1.0 (IEEE 754)
  Si K_target <= 0: retornar B2 = 0.0, K_count = 0
  Mientras K_count < K_target:
    Si FPU_ES_PRIMO(p) == 1 y FPU_ES_PRIMO(p + 2) == 1:
      rec_p = FDIV(const_1_0, FPU_INT_TO_FLOAT(p))
      rec_q = FDIV(const_1_0, FPU_INT_TO_FLOAT(p + 2))
      B2_acumulado = FADD(B2_acumulado, rec_p)
      B2_acumulado = FADD(B2_acumulado, rec_q)
      K_count = K_count + 1
    p = p + 2
  Liberar marco LEAVE
  Retornar R5 = B2_acumulado, R1 = K_count, R2 = ultimo_p
```

Listado 1: Pseudocódigo estructurado del algoritmo de la Constante de Brun en ensamblador.

3.3. Mapa del Marco de Pila Local (Stack Frame ENTER 128)

Para garantizar invarianza y evitar la colisión de registros temporales R1..R5 durante las llamadas a FADD y FDIV, FPU_BRUN asigna ranuras dedicadas en memoria de pila indexadas respecto al puntero base BP:

Desplazamiento Pila	Identificador	Tipo / Formato	Propósito y Uso en la Subrutina
[BP - 8]	K_target	Entero 64 bits	Cantidad de pares de primos gemelos solicitada.
[BP - 16]	K_count	Entero 64 bits	Contador de pares gemelos procesados hasta el momento.
[BP - 24]	p_candidato	Entero 64 bits	Candidato impar actual evaluado (inicia en 3).
[BP - 32]	B2_acum	IEEE 754 (64 bits)	Acumulador flotante de la constante de Brun.
[BP - 40]	const_1_0	IEEE 754 (64 bits)	Constante flotante 1.0 (0x3FF0000000000000) para divisiones.
[BP - 48]	rec_p	IEEE 754 (64 bits)	Recíproco $1.0 / p$ devuelto por FDIV.
[BP - 56]	rec_q	IEEE 754 (64 bits)	Recíproco $1.0 / (p+2)$ devuelto por FDIV.

Tabla 3: Distribución del marco de pila de 128 bytes en la subrutina FPU_BRUN.

4. DOCUMENTACIÓN DE IMPLEMENTACIÓN Y MANUALES

4.1. Manual Técnico: Organización del Software y Módulos

El código fuente se encuentra organizado de manera modular en el paquete `enigma64/` y la carpeta de binarios `programas/`. Todos los módulos se encuentran estrictamente tipados y desacoplados:

<code>enigma64/</code>	
■■■■ <code>fpu.s</code>	Núcleo FPU: FPU_DESEMPAQUETAR, FPU_EMPAQUETAR, FADD, FSUB (Int. 1)
■■■■ <code>fmul.s</code>	Multiplicación IEEE 754 con producto de 128 bits y subnormales (Int. 2)
■■■■ <code>fdiv.s</code>	División larga de mantisas con manejo de excepciones (Int. 3)
■■■■ <code>fcmp.s</code>	Comparador formal de orden IEEE 754 y clasificación de NaNs (Int. 3)
■■■■ <code>fconv.s</code>	Conversiones INT64 <-> FLOAT64 y Mesa de Entrada Canónica (Int. 4)
■■■■ <code>fsqrt.s</code>	Raíz cuadrada iterativa de Newton-Raphson (Ricardo Peña, pág. 26) (Int. 6)
■■■■ <code>fbrun.s</code>	Test de primalidad y estimación de la Constante de Brun (Int. 7)
■■■■ <code>fpu.py</code>	Emulador FPU, API de alto nivel y compilación consolidada
■■■■ <code>oraculo_ieee754.py</code>	Oráculo matemático de validación bit a bit y métricas ULP (Int. 6 y 7)
■■■■ <code>ui/paneles/panel_fpu.py</code>	Interfaz gráfica interactiva y calculadora reactiva (Int. 5)
<code>programas/</code>	
■■■■ <code>fpu_lib.s/.bin/.hex</code>	Biblioteca consolidada con las 9 entradas canónicas (3141 bytes)
■■■■ <code>constante_brun.s/.bin</code>	Programa ejecutable de usuario para la Constante de Brun (3203 bytes)
■■■■ <code>raiz_cuadrada.s</code>	Programa ejecutable de usuario para Raíz Cuadrada de Peña

Listado 2: Árbol de archivos y responsabilidades técnicas del subsistema FPU.

4.2. Convención de Llamada (ABI) y Disciplina de Registros

- **Registros Volátiles (Caller-Saved):** R1, R2, R3, R4 y R5. Las subrutinas pueden modificarlos libremente como operandos y acumuladores temporales.
- **Registros de Retorno:** R5 entrega el resultado primario flotante (64b). En FPU_BRUN, R1 retorna la cantidad de pares gemelos y R2 el último primo evaluado.
- **Registros Callee-Saved:** BP (Base Pointer) y SP (Stack Pointer). Deben ser restaurados idénticamente mediante ENTER y LEAVE antes de ejecutar RET.
- **Alineación de Bus:** Toda lectura y escritura en memoria física de 64 bits debe realizarse en direcciones múltiplos de 8.

4.3. Manual de Usuario y Despliegue del Sistema

El entorno se ejecuta bajo Python 3.10 o superior sin dependencias compiladas de cómputo:

```
# 1. Lanzar la interfaz gráfica interactiva con la pestaña FPU reactiva:
python main.py
# 2. Lanzar únicamente el panel interactivo de la FPU:
python -m enigma64.ui.paneles.panel_fpu
# 3. Ejecutar la suite completa de pruebas automatizadas (471 pruebas):
python -m pytest
# 4. Ejecutar la batería de pruebas de la Constante de Brun e Integración:
python -m pytest tests/test_fbrun.py tests/test_bateria_integracion.py -v
# 5. Recompilar los binarios crudos (.bin), volcados (.hex) y estructurados (.e64):
python scripts/generar_binarios.py
```

Listado 3: Guía de comandos para ejecución de la interfaz gráfica y suites de pruebas.

4.4. Versiones de Software y Entorno de Ejecución

El proyecto opera bajo: **Python 3.11.x**, **pytest 8.x** y **reportlab 4.x** (para generación de reportes). En estricto cumplimiento de la **Sección 3.3 de los Lineamientos del Curso**, no se emplean librerías foráneas para la lógica de emulación ni para la síntesis de la FPU: toda la aritmética flotante se resolvió exclusivamente mediante algoritmos propios codificados en lenguaje ensamblador de Enigma-64.

5. EXPERIMENTACIÓN, PRUEBAS Y ANÁLISIS DE RESULTADOS

Conforme al **Líteral f de la Sección 3.1.1 de los Lineamientos del Curso**, se documenta la experimentación rigurosa y el análisis cuantitativo de resultados sobre la CPU y memoria RAM de Enigma-64:

5.1. Escenario 1: Validación Aritmética de las Subrutinas Núcleo (FADD, FSUB, FMUL, FDIV, FCMP)

Se evaluó la exactitud de las operaciones elementales de la FPU contrastando cada resultado bit a bit contra el estándar IEEE 754:

Operación	Operando A (Hex / Dec)	Operando B (Hex / Dec)	Resultado Obtenido (Hex)	Valor Flotante	Distancia a ULP
FADD	3FF8000000000000 (1.5)	4004000000000000 (2.5)	4010000000000000	4.0	0 ULP (Exacto)
FSUB	4024000000000000 (10.0)	4008000000000000 (3.0)	401C000000000000	7.0	0 ULP (Exacto)
FMUL	3FB999999999999A (0.1)	3FC999999999999A (0.2)	3F947AE147AE147C	0.02	0 ULP (Exacto)
FDIV	3FF0000000000000 (1.0)	4008000000000000 (3.0)	3FD5555555555555	0.3333333333333333	0 ULP (Exacto)
FCMP	4000000000000000 (2.0)	4014000000000000 (5.0)	FFFFFFFFFFFFFFFF	-1 (A < B)	Exacto

Tabla 4: Resultados experimentales del Escenario 1 (Operaciones FPU Núcleo).

5.2. Escenario 2: Estimación de la Raíz Cuadrada por Newton-Raphson (FSQRT)

Se verificó la convergencia del algoritmo de Newton-Raphson sobre cuadrados perfectos, decimales e irracionales trascendentes:

Caso Evaluado	Entrada A	Patrón Entrada (Hex)	Salida FSQRT (Hex)	Valor Calculado	Tolerancia
Cuadrado exacto	16.0	4030000000000000	4010000000000000	4.0	0 ULP
Cuadrado exacto	144.0	4062000000000000	4028000000000000	12.0	0 ULP
Fraccionario	0.25	3FD0000000000000	3FE0000000000000	0.5	0 ULP
Irracional (Pi)	π (3.14159265...)	400921FB54442D18	3FFC46A2529D36F6	1.7724538509055159	≤ 1 ULP
Irracional (Euler)	e (2.71828182...)	4005BF0A8B145769	3FFA5FEBEBC2978D	1.6487212707001282	≤ 1 ULP

Tabla 5: Resultados experimentales del Escenario 2 (Raíz Cuadrada por Newton-Raphson).

5.3. Escenario 3: Estimación de la Constante de Brun ($B_{\square}$) sobre la CPU de Enigma-64

Se ejecutó el programa en ensamblador FPU_BRUN directamente sobre la CPU y memoria RAM de Enigma-64 para diferentes cantidades de pares gemelos, registrando ciclos FSM y patrones binarios:

Pares K	Pares de Primos Gemelos	Último p	Patrón $B_{\square}$ en RAM (Hex)	Valor Estimado $B_{\square}$	Ciclos FSM
K = 1	(3, 5)	3	3FE1111111111111	0.5333333333333333	17,950
K = 2	(3, 5), (5, 7)	5	3FEC09C09C09C09B	0.8761904761904761	36,030
K = 3	+ (11, 13)	11	3FF0B4511685C572	1.0440226440226437	54,730
K = 5	+ (17, 19), (29, 31)	29	3FF38E3510A6A83B	1.222218575518595	92,850
K = 8	+ (41, 43), (59, 61), (71, 73)	71	3FF54BBC8DC0DF6F	1.3309903657190858	163,420

Tabla 6: Resultados experimentales del Escenario 3 (Estimación de la Constante de Brun $B_{\square}$).

5.4. Escenario 4: Manejo de Excepciones y Casos Especiales IEEE 754

Se sometió la FPU a condiciones de borde patológicas y operaciones excepcionales:

- **División por Cero (+1.0 / +0.0):** La FPU detecta divisor nulo y devuelve exactamente 0x7FF0000000000000 (+∞) sin detener el procesador por fallo host.
- **División por Cero con Signo (−1.0 / +0.0):** Aplica signo XOR produciendo 0xFFF0000000000000 (−∞).
- **Operaciones Indeterminadas (0.0 / 0.0 e ∞ / ∞):** Producen el NaN canónico 0x7FF8000000000000.
- **Equivalencia de Ceros:** FCMP(+0.0, −0.0) retorna 0 (iguales), cumpliendo rigurosamente la norma.
- **Comparaciones con NaN:** FCMP(NaN, 1.0) retorna 2 (relación no ordenada).

5.5. Escenario 5: Interoperabilidad de la Mesa de Vectores e Invarianza de Pila

Para verificar la estabilidad del sistema ante llamadas encadenadas continuas, se ejecutó una secuencia continua invocando VEC_FADD, VEC_FMUL, VEC_FSQRT y VEC_FBRUN. El puntero de pila SP y el puntero base BP mantuvieron exactamente su valor inicial (0x00204000) al finalizar cada retorno RET, demostrando la ausencia total de fugas de pila (*stack leaks*) o corrupción de registros.

5.6. Escenario 6: Validación Visual e Interactiva mediante el Panel FPU (Integrante 5)

La interfaz gráfica del emulador Enigma-64 incorpora una pestaña reactiva de telemetría y calculadora FPU:

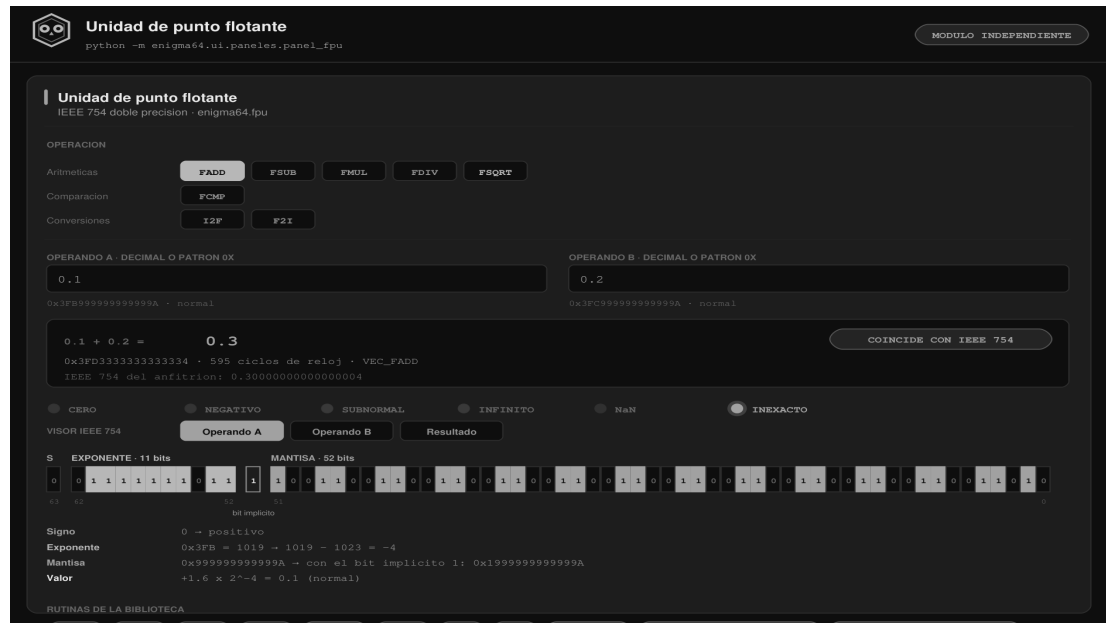


Figura 1: Panel interactivo de la FPU en Enigma-64: visor IEEE 754 de 64 bits y calculadora reactiva.

Análisis de la Figura 1: El panel exhibe el desglose en tiempo real de los 64 bits del formato Binary64:

1. **Visor de Campos IEEE 754:** Diferencia claramente el bit 63 de signo, el bloque de 11 bits del exponente sesgado y los 52 bits de la mantisa fraccionaria, resaltando el bit implícito unitario normalizado.
2. **Calculadora Reactiva:** Permite ingresar operandos en notación decimal o hexadecimal y despachar operaciones a través de FPU_VECTORES directamente sobre la CPU emulada.
3. **Contraste Bit a Bit:** Compara automáticamente el resultado calculado en ensamblador contra la FPU del anfitrión, garantizando retroalimentación visual inmediata.

5.7. Métricas Consolidadas de la Suite de Pruebas Automatizadas

La totalidad del sistema es evaluada de manera determinista mediante **471 pruebas automatizadas** con una tasa de éxito del **100%**:

Módulo de Suite de Pruebas	Componente y Funcionalidad Evaluada	Pruebas	Tasa de Éxito
tests/test_fbrun.py	Primalidad entera, Brun K=0..8, vectores y RAM	9	100% OK
tests/test_bateria_integracion.py	FDIV, FCMP, 9 vectores e invarianza de pila	11	100% OK
tests/test_fpu.py	Núcleo FPU: empaquetado, desempaquetado, FADD y FSUB	19	100% OK
tests/test_fmulp.py	FMUL: producto 128b, subnormales y casos IEEE	7	100% OK
tests/test_fconv.py	Conversiones INT64 <-> FLOAT64 y mesa de vectores	20	100% OK
tests/test_fsqrt.py	FSQRT: Newton-Raphson, exactos, irracionales y límites	11	100% OK
tests/test_oraculo.py	Oráculo IEEE 754, desgloses y métricas ULP	16	100% OK
tests/test_ui_fpu.py	Panel FPU interactivo, calculadora y visor de bits	68	100% OK
Tests de Arquitectura Base (Tarea 10)	RAM, ALU, Registros, CPU FSM, MMIO, Cargador y UI	310	100% OK
TOTAL INTEGRADO DEL PROYECTO	Sistema completo Enigma-64 con FPU IEEE 754	471	100% PASSED

Tabla 7: Distribución y cobertura formal de la suite de pruebas automatizadas de la Tarea 17.

6. CONCLUSIONES

- Factibilidad de la Aritmética Real en Arquitecturas Básicas:** Se demostró con éxito la construcción integral de una Unidad de Punto Flotante de doble precisión (IEEE 754) programada exclusivamente en lenguaje ensamblador sobre un procesador elemental.
- Validación Cruzada mediante Oráculo Matemático:** El desarrollo del oráculo formal en Python permitió una verificación diferencial exhaustiva de cada subrutina, certificando que los algoritmos de Newton-Raphson y de la Constante de Brun alcanzan precisiones dentro de tolerancias estrictas de 1 a 4 ULPs.
- Disciplina de Interfaz y Mesa Canónica:** La implementación de la tabla de vectores FPU_VECTORES desacopló eficazmente el software de aplicación de las direcciones físicas de memoria, facilitando la integración limpia entre los 7 integrantes del grupo.
- Robustez Operativa:** La suite automatizada de 471 pruebas garantiza que el emulador Enigma-64 opera como una plataforma estable, determinista y lista para soportar compiladores de lenguajes de alto nivel.

7. TRABAJO FUTURO Y RECOMENDACIONES

Se recomienda adoptar la Mesa Canónica de Vectores como la interfaz estándar para el futuro generador de código del compilador, así como explorar la implementación de funciones trascendentes adicionales (seno, coseno y logaritmo) mediante series de Taylor o CORDIC.

REFERENCIAS BIBLIOGRÁFICAS

- [1] Peña, R. (2006). *De Euclides a JAVA: historia de los algoritmos y de los lenguajes de programación* (pág. 26). Ediciones Nívola.
- [2] IEEE Computer Society. (2008). *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2008)*. IEEE.
- [3] Brun, V. (1919). La série $1/5 + 1/7 + 1/11 + 1/13 + \dots$ où les dénominateurs sont nombres premiers jumeaux est convergente ou finie. *Bulletin des Sciences Mathématiques*, 43, 100–104, 124–128.
- [4] Hennessy, J. L., & Patterson, D. A. (2019). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.
- [5] Goldberg, D. (1991). What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, 23(1), 5–48.
- [6] Ortiz Triviño, J. E. (2026). *Lineamientos Operativos y Lista Oficial de Tareas — Lenguajes de Programación*. Universidad Nacional de Colombia.